

Q1. WHAT IS GITGO?

Simple Definition: A VS Code extension that publishes coding-problem solutions to GitHub using one command.

Manual Process (High Friction)

Create folder → Copy/rename solution → Write README → Link screenshots → Git add → Git commit → Push to remote → Create PR

GitGo Workflow (One Action)

Run "GitGo: Publish Solution" → Full modular automation pipeline handles execution, Git tasks, and remote sync.

Interview Response: "GitGo is a productivity-focused VS Code extension. It replaces a manual, 8-step solution-publishing ritual with a single command, automating file-system structures, README generation, screenshot attachment, and Git operations (PR or direct push)."

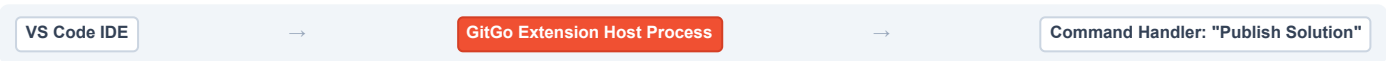
Q2. WHY DID WE BUILD IT?

Motivation: Git itself is not the problem; the issue is the repetitive chore of documenting and organizing code solutions. Developers often skip publishing solved DSA problems because of the steps involved. GitGo merges these into a single controlled pipeline.

Key Principle: One Command = One Complete Publish Pipeline

Q3. WHAT IS A VS CODE EXTENSION?

Simple Definition: A program running inside VS Code's process boundary to add new commands, configurations, or UI features.



GitGo runs inside the isolated VS Code Extension Host Process, ensuring it doesn't block the editor's main UI thread.

Q4. HOW DOES THE GITGO COMMAND WORK INTERNALLY?

Flow: When triggered, the VS Code command router activates GitGo, executing its registered command handler.

- Activation: Triggered by command IDs mapped in package.json under activationEvents.
- Command Registration: Hooked in src/extension.ts via vscode.commands.registerCommand.
- Execution Handler: delegates logic to src/commands/publishSolution.ts, which requests inputs and calls the pipeline.

Q5. WHAT HAPPENS AFTER I CLICK "PUBLISH SOLUTION"?

Sequence of Actions: GitGo executes a linear orchestration flow to prepare, write, and upload the solution.

- 1 Get active file path:Resolves path and detects language (src/services/languageDetector.ts)
- 2 Collect user input:Prompts for problem type, difficulty, push mode, screenshots, and custom folder name
- 3 Build Context:Runs buildPublishContext to encapsulate configuration data (PublishContextBuilder.ts)
- 4 Directory stage:Creates the destination problem folder structure on disk (folderCreator.ts)
- 5 Solution stage:Copies and renames the source solution file to standard format (fileCopier.ts)
- 6 README stage:Generates and writes documentation containing metadata (readmeGenerator.ts)
- 7 Screenshot stage:Copies selected screenshots and embeds links in the README (screenshotHandler.ts)
- 8 Git stage:Stages changes, commits locally, and pushes branch to remote (gitService.ts)
- 9 PR or Push execution:If PR mode, copies description to clipboard and opens the GitHub compare page in browser
- 10 Dashboard Sync:Regenerates progress stats and inserts the summary table back into the main README

Q6. WHAT IS PUBLISHCONTEXT?

Simple Definition: A single state object passed through the entire publishing pipeline containing all resolved metadata. Instead of passing 10+ arguments down, the pipeline updates and references this single structured type defined in src/pipeline/PublishContext.ts :

```
{ request, language, standardFileName, author, repoPath, destinationFolder, repositoryInfo, screenshots, codeContent, createdDirectory, createdReadme, createdBranch, branchPushed, commitHash }
```

Q7. WHAT IS THE PUBLISH PIPELINE?

Simple Definition: A sequential workflow controller that runs multiple stages in order. Controlled by the orchestrator PipelineExecutor.ts, it maps stages directly to dedicated services:



Q8. WHY MODULAR ARCHITECTURE? WHY NOT PUT EVERYTHING IN ONE FUNCTION?

Answer: Each part of the publishing flow has a distinct, single responsibility. Keeping them separated into stages makes the code base highly maintainable, testable, and robust.

- **File Systems logic** (`DirectoryStage` / `SolutionStage`) is isolated from Git logic.
- **Doc Generation logic** (`ReadmeStage`) does not depend on remote network conditions.
- **Single Responsibility Principle (SRP):** Changes in Git commands or README layouts won't break file copy utilities.

Q9. HOW DOES GIT AUTOMATION WORK INTERNALLY?

Answer: GitGo executes native Git commands using a custom execution runner (`GitCommandRunner.ts`) which wraps Node's `child_process.spawnSync` with strict timeout and error-parsing mechanisms.

Normal Push Mode Flow

1. Checkout default branch (`git checkout <branch>`)
2. Pull remote updates (`git pull origin <branch>`)
3. Stage files (`git add .`)
4. Commit locally (`git commit -m "..."`)
5. Push to branch (`git push origin <branch>`)

Pull Request Mode Flow

1. Clean old local/remote branch if any
2. Create feature branch (`git checkout -b <branch>`)
3. Stage and commit changes
4. Push branch (`git push -u origin <branch>`)
5. Copy PR desc to clipboard & open PR Compare page

Q10. HOW DOES DEFAULT BRANCH DETECTION WORK?

Answer: GitGo uses the `defaultBranchDetector.ts` module to query the repository dynamically rather than assuming "main" or "master":

1. Tries to read symbolic reference: `git symbolic-ref refs/remotes/origin/HEAD`
2. Queries remote metadata: matches "HEAD branch" output from `git remote show origin`
3. Checks local lists: searches local branch lists for common names (`main` , `master` , `develop`)
4. Resolves current active branch: `git rev-parse --abbrev-ref HEAD`
5. Ultimate fallback: Defaults to `"main"` if all else fails.

Q11. WHAT DOES ATOMIC / TRANSACTIONAL EXECUTION MEAN HERE?

Concept: "All-or-nothing" execution. If a stage fails (e.g. network push fails), the repository must not be left in a corrupted or partially modified state.

Start Pipeline → Execute Stages → Stage Fails X → Rollback Executed Stages (Reverse)

The `PipelineExecutor` records executed stages. If any stage errors out, it calls their `rollback()` methods in exact reverse order.

Q12. HOW DOES ROLLBACK WORK IF SOMETHING FAILS?

Implementation: The codebase handles localized rollback hooks to clean up local and remote environments:

- **Directory Cleanup:** If `DirectoryStage` created a new folder, it deletes the folder recursively using node `fs.rmSync`.
- **Git Reset (Normal Push):** If local commit was created but push failed, it runs `git reset --hard HEAD~1` to undo the commit.
- **Git Cleanup (PR Mode):** Checkouts back to base branch, deletes local feature branch (`git branch -D`), and attempts remote deletion (`git push origin --delete <branch>`).
- **Limitations:** GitGo does not use GitHub APIs directly for PRs, meaning no remote Pull Request object is ever left dangling to delete.

Q13. HOW DO SETTINGS AND MEMORY WORK?

Implementation: Saved locally using VS Code's global persistent configuration storage. When a user runs the command, GitGo queries the `vscode.workspace.getConfiguration("gitgo")` API. If config fields like `author.name`, `repoPath`, or `lastParentFolder` are missing, it prompts the user and saves the inputs globally back to configuration settings, preventing repetitive entries.

Q14. PERFORMANCE: HOW DID WE HANDLE 10,000+ PROBLEMS UNDER 2.3 SECONDS?

Verified Benchmark Methodology: This benchmark measures the **"Progress Dashboard Scanner"** (`DashboardScanner.ts`). It scans the target repo directory for DSA folders, parses problem counts, matches difficulties, and formats the table. The scan runs in linear $O(N)$ time by inspecting leaf-folder directories directly, skipping deeper node modules, parsing paths concurrently, and writing updates via regex replacements. It completes a 10,000-problem folder scan in **"2.29 seconds"** (speed of 1,200+ folders/sec).

★ INTERVIEW RAPID FIRE ANSWERS

What is GitGo?

A VS Code extension automating coding-problem solution publishing.

What is the Publish Pipeline?

A sequence of stages executing directory, file, readme, and git tasks.

Why modular architecture?

SRP: Separates file system, markdown, validation, and git dependencies.

How does rollback work?

Deletes generated folders and does hard git resets / remote branch deletes.

Why default branch detection?

Repos may use main/master/develop dynamically; symbolic-ref checks this.

Why did you build it?

To consolidate 8 manual copy/git steps into a single CLI flow.

What is PublishContext?

A consolidated state object holding all metadata across pipeline stages.

What is atomic execution?

All-or-nothing pipeline: failsafe rollback of local file and branch writes.

Push vs Pull Request?

Push goes to default branch; PR pushes feature branch & opens browser compare.

What does 10,000 / 2.3s mean?

Progress dashboard scanner benchmark speed, traversing 10k folders linearly.